

Senior Backend Engineer — skills assessment

take-home project

A focused coding exercise to assess real-world backend judgment. Complete it in your own environment and submit a link to your repository.

Time estimate	30–45 min
Stack	Python · FastAPI
Deliverable	GitHub repo

Focus on correctness, clarity, and judgment — not on building every feature. A small, well-reasoned solution beats a sprawling one.

Context

You are building a small internal **task management API** for a product team. The service lets users create, list, and update tasks. It doesn't need a UI — just a working HTTP API backed by an in-memory store (no database required for this exercise).

We care less about completeness and more about **how** you write code: structure, validation, error semantics, and the trade-offs you make under time pressure.

Requirements

Task 1 — core API FastAPI · Pydantic

Implement a FastAPI application with the following three endpoints:

POST	/tasks	Create a new task
GET	/tasks	List all tasks (support <code>?status= filter</code>)
PATCH	/tasks/{id}	Update a task's title or status

- A task has: `id`, `title`, `status` (`todo` | `in_progress` | `done`), `created_at`.
- Use Pydantic models for request validation and response serialization.
- Return appropriate HTTP status codes and structured error responses.
- Return `404` with a clear message if a task ID does not exist.

Task 2 — tests pytest · httpx

Write tests covering the key behaviours. You don't need 100% coverage — test what matters.

- At minimum: create a task, list with filter, update status, `404` on missing ID.
- Use `pytest` and FastAPI's `TestClient` or `httpx.AsyncClient`.
- Tests must pass with a single `pytest` command from the repo root.

Task 3 — write-up & engineering judgment **Judgment** · **Trade-offs**

Add a short `NOTES.md` (about one page max). Keep each answer tight — we're looking for reasoning and prioritization, not essays.

Design & trade-offs

- Any design decisions you made and why.
- What you would add or change given more time.
- One thing you deliberately left out — and why that was the right call.

Scaling work off the request path

- Suppose one of these operations grows into a long-running job — say marking a task `done` has to fan out notifications, recompute aggregates, or call a slow third-party service, and it can no longer finish inside the request/response cycle without making the client wait. How would you handle that? Walk us through your approach: where the work goes, how the endpoint stays responsive, and the trade-offs of the option(s) you'd reach for.

Security & performance

- This is an in-memory toy today. As it grows into a real service taking production traffic, what would you put in place around **security and performance**? Tell us what you'd reach for and why.

Production readiness

- What would you want in place **before this service goes to production**? Call out what matters most and what you'd do first.

What we're not looking for

- A real database — an in-memory dict or list is fine.
- Auth, rate-limiting, or other production hardening **in the code** — we ask you to *discuss* these in the write-up, not build them.
- Docker, CI config, or deployment setup.
- Pagination beyond basic filtering.

Scoring rubric

Area	What we look for
------	------------------

API correctness	Endpoints behave as specced; status codes and error shapes are correct; filter works.
Pydantic usage	Models are clean; validation errors surface as structured responses; no raw dicts passed around.
Code quality	Readable structure, sensible naming, no unnecessary complexity; code reads like a senior engineer wrote it.
Tests	Key paths covered; tests are clear and self-contained; all pass clean.
Write-up & engineering judgment	Reasoned trade-offs over buzzword lists; offloads long-running work sensibly (durability, retries, idempotency); names the security/performance and production-readiness work that actually matters and prioritizes it.

Submission

Push your solution to a public (or private, shared with us) GitHub repository. Include a `README.md` with setup and run instructions. A reviewer should be able to clone and run `pytest` within two minutes.

Submit the link within the agreed deadline. We will review asynchronously and follow up within two business days.